

Learning Conway’s Game of Life with CNN-Transformer Hybrids: A Project Summary

Hao-Yuan

August 25, 2026

Abstract

We summarize a project studying how CNN-Transformer hybrid models learn the one-step transition rule of Conway’s Game of Life (GoL) on a 40×40 toroidal grid. Across ten model versions we identify and fix two distinct failure modes — exposure bias under autoregressive rollout, and a data-scaling regression hidden from aggregate validation metrics — and arrive at a 1.5M-parameter model with zero errors on a 2,500-grid stress test. We independently verify a recent claim that polynomial activations have a strong inductive-bias advantage over ReLU for learning GoL from very few parameters, and we present an embedding-space experiment showing that sinusoidal positional encodings are empirically closer to translation-equivariant than learned ones. Full details, all code, and a 20-page extended writeup are available in the project repository.

1 Problem and Architecture

Conway’s Game of Life is a deterministic cellular automaton: each cell’s next state depends only on its own state and the count of live cells among its 8 neighbors (B3/S23 rule). We treat one GoL step as a supervised image-to-image translation problem on a 40×40 periodic (toroidal) grid, and ask a neural network to predict the next grid state from the current one.

Our core architecture (*CNNTransformerV4*, used from Task 12 onward) is a CNN-Transformer hybrid: two circular-padded convolutional layers extract local features respecting the torus topology, which are then cut into non-overlapping 4×4 patches, linearly projected, combined with a *learned* absolute positional embedding, passed through a Transformer encoder to mix long-range information, and finally projected back to per-cell logits. The convolutional stage guarantees translation-equivariant local feature extraction by construction (weight sharing plus circular padding); the Transformer stage adds global context that a purely convolutional model of small receptive field cannot, at the cost of the clean equivariance guarantee (Section 3).

2 Training Evolution: Two Failure Modes and Their Fixes

Exposure bias (V5 $\rightarrow$ V7). A model trained with pure single-step teacher forcing (V5, val. loss 0.201, precision/recall $> 99\%$) looks excellent on held-out single steps, but collapses when run autoregressively: small errors compound because the model never sees its own (imperfect) outputs during training. We first tried a straight-through-estimator (STE) multi-step scheme (V6) to expose the model to its own rollout during training with a differentiable path; this *diverges* after epoch 5 (val. loss rises from 0.234 to > 1.0), which we trace to a systematic gradient bias introduced by the STE’s hard threshold. Replacing STE with *scheduled sampling* (V7) — gradually increasing the probability of feeding the model its own previous prediction instead of the ground truth — fixes this cleanly: val. loss 0.201, precision 99.7%, recall 100%, stable across training.

A hidden data-scaling regression (V8 $\rightarrow$ V10). Upsampling high-density configurations $3\times$ (V8) improves per-grid accuracy further (100% precision, 291K params). Naively scaling the *training set* $5\times$ (V9) leaves aggregate validation metrics statistically unchanged (val. loss 0.201, precision 99.7%) — but a density-stratified failure search (500 random grids at each of five densities, same seed across models) reveals that V9’s per-grid error count at density 0.80 is nearly *seven times worse* than V8’s (22,190 vs. 3,358 summed false positives/negatives). This regression is invisible in aggregate metrics and only surfaces under targeted subgroup auditing. Instead of more data, we scale the *model*: V10 (128-dim, 6-layer Transformer, 1.5M params, $5.15\times$ V8’s size) trained from scratch achieves **zero errors** across all 2,500 test grids at every density (Figure 1). The practical lesson: once a model saturates its capacity, more data can silently trade average-case performance for worse tail-case performance, and only per-subgroup error auditing (not aggregate validation loss) can detect it.

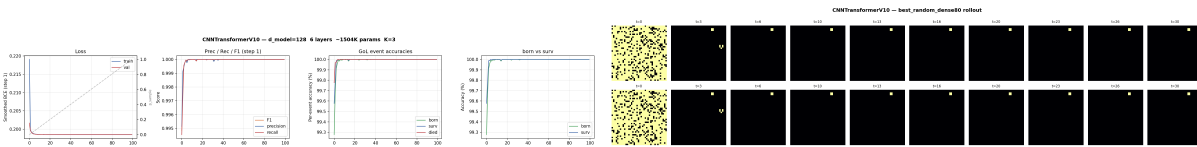


Figure 1: V10 training curve (left) and a 50-step autoregressive rollout at density 0.80 (right), the hardest test condition. V10 makes zero errors across 2,500 held-out grids spanning five densities.

Ver.	Key change	Val. loss	Precision/Recall	Status
V5	Single-step teacher forcing	0.201	99.4% / 99.7%	Collapses under rollout
V6	+STE multi-step	0.234 ^a	92.7% / —	Diverges
V7	STE $\rightarrow$ scheduled sampling	0.201	99.7% / 100%	Stable
V8	+ $3\times$ high-density upsample	0.201	100% / 99.9%	Best per-grid (291K params)
V9	+ $5\times$ all data	0.201	99.7% / 99.6%	Regressed tail errors
V10	$5.15\times$ model size	0.199	100% / 100%	Zero errors , 2,500/2,500

Table 1: Version summary (abbreviated; see extended writeup for the full V1–V11 table and per-density error breakdown). ^aV6’s best epoch before divergence.

3 Embedding Geometry: Sinusoidal vs. Learned Positional Encoding

A CNN’s feature map is a *physical* embedding: because convolution is translation-equivariant by construction, shifting the input shifts the feature map by exactly the same amount, with the same content vector at each location. A Transformer’s patch embeddings, after a learned absolute positional embedding is added, have no such architectural guarantee — each position slot is an independently learned vector with no structural relationship to its neighbors.

We tested whether this shows up empirically by comparing two of our trained models that differ (among other things) in positional encoding: V2 uses a fixed 2D sinusoidal positional encoding (whose angle-addition identities give it a closed-form near-equivariant structure under translation), while V8 uses a learned absolute positional embedding. We measured cosine similarity between patch embeddings of a grid and of the same grid shifted by several offsets, for both models, across glider and random configurations (Figure 2). V2’s post-Transformer embeddings stay far more similar

under translation (mean cosine similarity 0.997) than V8’s (0.890), consistent with the theoretical prediction that sinusoidal encodings impose a more equivariant geometry on the representation space. (We note an acknowledged confound: V2 uses zero-padding in its convolutional stage versus V8’s circular padding, so this is not a perfectly isolated ablation of positional encoding alone — see the extended writeup for a proposed fully controlled follow-up.)

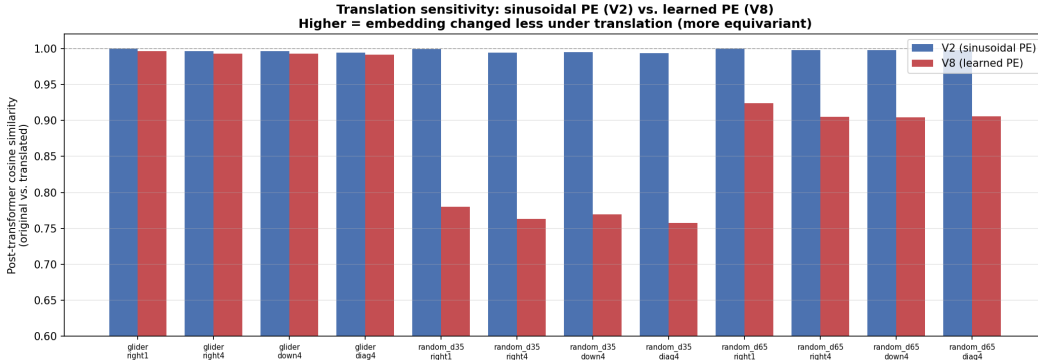


Figure 2: Post-Transformer embedding cosine similarity under translation, sinusoidal PE (V2) vs. learned PE (V8). Higher means the embedding changed less under translation, i.e. more equivariant.

4 Independent Verification: Polynomial Activations for GoL

Motivated by a recent paper (Ahmed & Davis, 2026, arXiv:2606.23587) claiming that a learnable second-degree polynomial activation $\phi(x) = w_0 + w_1x + w_2x^2$ lets a minimal CNN learn GoL reliably where ReLU fails almost completely, we independently reproduced this claim from scratch rather than taking it on faith. We built two CNNs matching the paper’s minimal $L(1, 1)$ architecture (two 1×1 convolutions plus one 3×3 circular-padded convolution) that are **identical except for the activation function**, exactly matching the paper’s reported parameter counts (ReLU: 25 params, polynomial: 34 params). Trained single-step, teacher-forcing, on a small dataset (100 train / 20 val. grids) over 10 random seeds each:

- **ReLU:** 1/10 seeds converge to perfect validation accuracy; mean final accuracy 89.4%.
- **Polynomial:** 10/10 seeds converge; mean final accuracy 100.0%.

This confirms the paper’s central claim on our own toroidal GoL setup (Figure 3). We then attempted to transfer this activation into our larger multi-step CNN-Transformer model (V11, same size as V8) by replacing GELU with polynomial activations in the CNN stage. This **failed**: training diverged sharply after epoch 5 under the scheduled-sampling regime, likely because the unbounded w_2x^2 term compounds through the autoregressive feedback loop in a way it cannot in the paper’s single-step setting. We report this as a negative result; candidate fixes (a bounded/saturating polynomial variant, a separate learning rate for polynomial coefficients, verifying single-step stability before adding rollout) were identified but not yet pursued.

5 Conclusion

Three practical lessons emerge from this project. First, autoregressive exposure bias in sequence/rollout models is best fixed with scheduled sampling rather than a hard differentiable

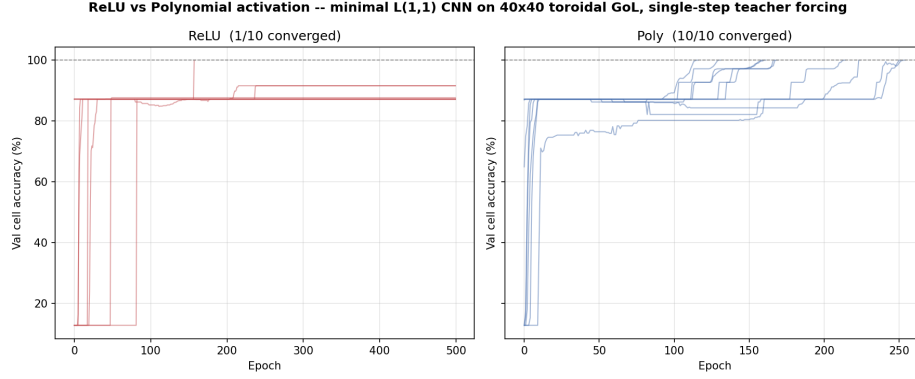


Figure 3: Validation accuracy over training, 10 seeds per activation, minimal 25/34-parameter CNN on 40×40 toroidal GoL. Polynomial activation converges reliably; ReLU converges in only 1 of 10 seeds.

relaxation (STE), which can introduce its own destabilizing gradient bias. Second, aggregate validation metrics can hide severe, density-specific regressions; per- subgroup error auditing is necessary once a model is scaled with more data. Third, an activation function’s inductive-bias advantage demonstrated at minimal scale and single-step training does not automatically transfer to a larger, multi-step, autoregressively-trained model — the interaction between an activation’s stability properties and the training regime matters as much as the activation itself.

Full code, all intermediate model checkpoints, the complete V1–V11 version history, additional embedding-space visualizations (rotation, reflection, cell-perturbation transforms), and a fully detailed 20-page writeup are available in the project repository ([game_of_life_demo/](#)).